CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

Arquitectura orientada a microservicios

Arquitectura y Diseño de Software · Jesús Ariel González Bonilla · 2026-B · Corte 1 · Unidad 2

Descomposición y comunicación entre componentes

Partir un sistema es fácil; partirlo por donde toca es el oficio

> **Un corte mal puesto se paga en cada funcionalidad nueva, durante años.**

- Cualquiera puede dividir en seis servicios
- Casi nadie acierta dónde va la frontera
- Y la frontera es lo más caro de mover

Lo que vas a poder hacer al terminar la unidad

- Verificar si algo es de verdad un microservicio
- Decidir dónde va el corte y defenderlo
- Elegir entre síncrono y asíncrono con criterio
- Distinguir un evento de un comando
- Mantener integridad sin transacción distribuida
- Reconocer el monolito distribuido a tiempo

01

1. Qué es un microservicio

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

Tres criterios que sí se pueden verificar

Despliegue independiente

¿Puedo liberar sin coordinar con otro equipo?

Datos propios

¿Alguien más lee o escribe mi tabla?

Capacidad de negocio

¿Puedo nombrarlo sin decir «capa» ni «módulo de»?

Si falla uno

Todavía no es un microservicio, es otra cosa

Lo que sí trae el estilo, y de qué depende

Beneficio	Solo llega si ya tienes
Escalado individual	Carga **desigual** entre capacidades
Despliegue independiente	CI/CD y pruebas de contrato
Resiliencia ante fallos	Timeouts, reintentos, circuit breaker
Autonomía de equipos	Equipos que ya existen y son dueños
Libertad tecnológica	Capacidad de operar varias pilas a la vez
Diagnóstico de fallas	Trazado distribuido

La prima se paga completa desde el primer día

Equipo pequeño, dominio incierto

- El costo llega ya
- Los beneficios no llegan
- Gana el monolito modular

Organización grande, dominio estable

- El costo se absorbe
- El escalado y la autonomía pagan
- Gana distribuir

El caso Netflix, leído como decisión

Monolito de streaming

Cambios lentos, fallos globales

Crecimiento global

Escalabilidad y confiabilidad al límite

Migración a microservicios

Cada componente con su base y su despliegue

Resultado

Escala, entrega rápida, fallo aislado

PREGUNTA

¿Cuál de estos es un PRERREQUISITO para adoptar microservicios, más que un beneficio?

A Escalar cada componente de forma independiente

B Usar una tecnología distinta en cada servicio

C Contar con despliegue automatizado y observabilidad distribuida

D Reducir el acoplamiento entre capacidades de negocio

02

2. Dónde va el corte

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

El corte se decide por negocio, no por capas técnicas

Corte técnico

- servicio-web, servicio-lógica, servicio-bd
- Un cambio de negocio toca los tres
- Pagas la red sin ganar nada

Corte por capacidad

- Control de acceso, Tarifación, Facturación
- Un cambio de negocio toca uno
- Escalas y despliegas por separado

Cinco formas de decidir dónde poner la frontera

Heurística	Corta por	Ejemplo en el parqueadero
Capacidad de negocio	Lo que la organización hace	Acceso · Tarifación · Facturación
Contexto delimitado	Dónde cambia el significado	«Vehículo» en acceso ≠ en facturación
Razón de cambio	Lo que se modifica junto	Tarifas vs. integración de cámaras
Volatilidad	Lo que cambia mucho vs. lo estable	Promociones vs. registro de ingresos
Perfil de carga	Lo que escala distinto	Reportería vs. operación
Autonomía de equipo	Quién es dueño de qué	Un servicio por equipo responsable

Cómo se llega al corte, en la práctica

1

Lista los eventos

Vehículo ingresó, tarifa calculada, pago recibido, cupo liberado

2

Agrupa

Los que comparten vocabulario y reglas

3

Nombra

Con un sustantivo de negocio. Si no sale natural, el grupo está mal

4

Verifica los datos

Cada grupo debe poder ser dueño de sus tablas

5

Cuenta los saltos

Más de tres servicios en un caso frecuente, revisa el corte

El antipatrón hermano: el *entity service*

Un servicio que solo hace CRUD sobre una tabla y al que todos los demás llaman en cadena no es un microservicio: es una base de datos con HTTP delante.

- **No encapsula una capacidad**
- **Nadie puede cambiarlo sin romper a otros**
- **Multiplica los saltos de red sin aislar nada**

La prueba del cambio derrumba la mitad de las propuestas

1

Enuncia un cambio real

«Agregar tarifa nocturna para motos»

2

Cuenta los servicios

¿Cuántos hay que tocar y desplegar?

3

Si es más de uno

El corte no siguió la razón de cambio

4

Corrige la frontera

Antes de escribir una línea de código

PREGUNTA

Un equipo divide el sistema en "servicio-api", "servicio-negocio" y "servicio-datos". ¿Qué antipatrón aplicó?

A Base de datos compartida entre servicios

B Descomposición por capas técnicas en lugar de capacidades

C Coreografía de Saga mal implementada

D Comunicación síncrona en cadena

03

3. Cómo se hablan los servicios

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

La pregunta no es el protocolo: es si esperas respuesta

	UNO A UNO	UNO A MUCHOS
SÍNCRONO (espero)	Request / Response REST, gRPC	(no aplica)
ASÍNCRONO (no espero)	Comando por cola AMQP, SQS	Publicar / Suscribir eventos, tópicos

Qué cambia según el estilo que elijas

	Síncrono (REST, gRPC)	Asíncrono (mensajería, eventos)
Quien llama	Espera la respuesta	Continúa de inmediato
Acoplamiento temporal	**Alto**: ambos deben estar arriba	**Bajo**: el receptor puede estar caído
Si el destino falla	El llamador falla o degrada	El mensaje espera en la cola
Consistencia	Inmediata	Eventual
Depuración	Una traza lineal	Requiere trazado distribuido
Uso típico	Consultas, validaciones en línea	Notificaciones, proyecciones

REST, mensajería y protocolos asincrónicos

REST

HTTP, sin estado, recursos por URL. Simple y universal. Su límite es el acoplamiento temporal

Mensajería

El emisor deja el mensaje y sigue; el consumidor lo toma cuando puede. Absorbe picos

AMQP

Mensajería empresarial con enrutamiento. Es lo que usa RabbitMQ

MQTT

Para dispositivos con poco ancho de banda. Típico en IoT y sensores de placa

Un evento narra un hecho; un comando da una orden

EVENTO: ALGO QUE YA OCURRIÓ

```
eventos.publicar(new SalidaRegistrada(ticket.id(), Instant.now()));
// cero, uno o muchos suscriptores. El emisor no sabe quiénes son.
```

CONSUMIDOR IDEMPOTENTE

```
@RabbitListener(queues = "salidas")
public void al(SalidaRegistrada evento) {
    if (yaProcesado(evento.id())) return;     // el mismo mensaje puede llegar dos veces
    notificador.enviarRecibo(evento.ticketId());
    marcarProcesado(evento.id());
}
```

Publicar comandos disfrazados de eventos reintroduce exactamente el acoplamiento que se quería eliminar.

Fallar bien: lo mínimo en comunicación síncrona

Mecanismo	Qué hace	Cuidado
Timeout	Corta la espera y libera el hilo	Sin él, un servicio lento propaga la caída
Reintento con backoff	Supera fallos transitorios	Solo para operaciones idempotentes
Circuit breaker	Deja de llamar a un destino que falla	Necesita una respuesta alternativa definida

PREGUNTA

El cobro en la talanquera debe confirmarse antes de abrir la barrera. ¿Qué estilo de comunicación corresponde?

A Síncrono, porque el resultado condiciona el paso siguiente

B Asíncrono, para desacoplar la pasarela del core

C Publicar/suscribir, para que varios servicios reaccionen

D Indiferente: ambos estilos sirven igual aquí

PREGUNTA

¿Qué distingue un evento de un comando?

A El evento viaja por cola y el comando por HTTP

B El comando es asíncrono y el evento es síncrono

C El evento narra un hecho ocurrido; el comando ordena una acción a un destinatario

D El evento siempre tiene varios suscriptores y el comando ninguno

04

4. Los datos: el problema difícil

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

Compartir la base de datos anula el estilo completo

Dos servicios, una base

El esquema es un contrato oculto

Un ALTER TABLE

Rompe a los dos a la vez

Hay que desplegar juntos

Adiós autonomía

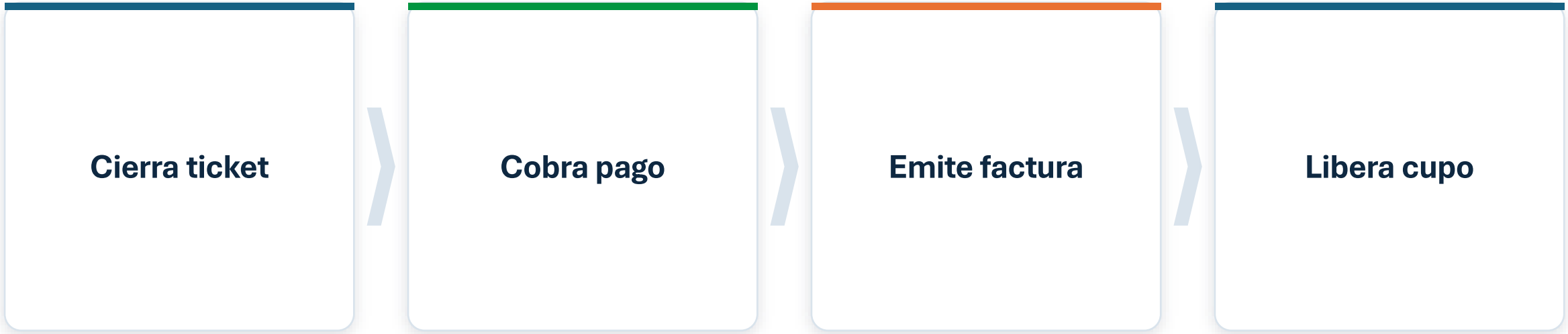
Monolito distribuido

Todos los costos, ningún beneficio

Consistencia fuerte y consistencia eventual

Consistencia fuerte	Consistencia eventual
Todos ven el mismo dato al instante	Convergen tras un intervalo
Requiere transacción compartida	Funciona con servicios independientes
Escala mal entre nodos	Escala bien
«El cupo se descontó ya»	«El recibo llega en unos segundos»

En una Saga, cada paso define su compensación



No existe rollback entre bases independientes: la compensación no deshace el pasado, agrega un hecho que anula su efecto.

Coreografía u orquestación: quién dirige la Saga

	Coreografía	Orquestación
Quién decide	Cada servicio reacciona a eventos	Un coordinador dirige los pasos
Acoplamiento	Mínimo	El orquestador conoce el flujo
Visibilidad del flujo	Disperso, difícil de leer	Centralizado y explícito
Recomendable para	2–3 pasos	Flujos largos o con reglas complejas

Guardar en la base y publicar el evento son dos operaciones

1

El problema

Guardas bien, la publicación falla, nadie se entera

2

Outbox

Guardas el dato Y el evento en la MISMA transacción

3

Un proceso lee la outbox

Y publica, reintentando hasta confirmar

4

El precio

El evento puede salir dos veces. Consumidores idempotentes

PREGUNTA

¿Cuál es la consecuencia directa de que dos servicios compartan la misma base de datos?

A Mejora la consistencia y por eso es una buena práctica

B Obliga a implementar Saga entre ambos servicios

C Se pierde el despliegue independiente: un cambio de esquema los afecta a ambos

D Aumenta la latencia de red entre los servicios

PREGUNTA

Una Saga falla en el tercer paso, después de haber emitido la factura. ¿Qué corresponde hacer?

A Ejecutar un rollback distribuido sobre los tres servicios

B Ejecutar la compensación definida: emitir una nota crédito

C Reintentar el paso indefinidamente hasta que tenga éxito

D Marcar la operación como inconsistente y escalarla a soporte

PREGUNTA

¿Qué problema resuelve el patrón Outbox?

A Evita que el mismo mensaje se entregue dos veces

B Que la escritura en la base y la publicación del evento no queden desacopladas

C Reduce la latencia entre servicios asíncronos

D Permite compartir base de datos sin acoplar servicios

05

5. Errores, cierre y trabajo de la unidad

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

Los errores que producen el monolito distribuido

1. Cortar por capas técnicas en vez de capacidades

2. Compartir la base de datos entre servicios

3. Servicios CRUD anémicos que solo envuelven una tabla

4. Llamadas en cadena síncronas: cinco saltos, cinco riesgos

...y los que aparecen después, en operación

5. Publicar comandos como si fueran eventos

6. Consumidores no idempotentes que duplican efectos

7. Omitir timeouts, y que un servicio lento arrastre a los demás

8. Adoptar el estilo sin CI/CD ni observabilidad

9. Empezar por microservicios en un dominio que no se entiende

Del concepto al foro: cuatro piezas de una intervención sólida

Pieza	De dónde sale
Una descomposición propuesta, con sus límites	Sección 2
La justificación del corte (heurística usada)	Sección 2, las cinco heurísticas
El estilo de comunicación de cada relación	Sección 3
Cómo se mantiene la integridad entre servicios	Sección 4

Tres ideas para llevarte

El corte manda

Por capacidad de negocio, nunca por capa técnica

Lo síncrono acopla

La pregunta es si esperas respuesta

Repartir datos mueve el problema

Saga, compensación, idempotencia

CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

¿Preguntas?

Jesús Ariel González Bonilla · ariel5253@hotmail.com